

Regression & Forecasting Under Distribution Shift

1 Problem Overview

This project studies supervised prediction under multiple data-generation settings that differ in structure and difficulty. Each row contains a feature vector and a real-valued target y . The goal is to learn a predictor $\hat{y} = f(\cdot)$ that generalizes well to the held-out test set.

The benchmark is divided into four parts (A–D), each targeting a different modeling capability:

- Part A: mostly linear signal with correlated covariates.
- Part B: localized nonlinear structure that requires interaction-sensitive modeling.
- Part C: panel forecasting with temporal dependence and lag features.
- Part D: panel forecasting under distribution shift and outlier contamination.

Participants may train either a single global model or separate part-specific models selected using the `part` column at inference time. Both approaches are allowed.

2 Quick Start

This competition requires participants to predict the real-valued target y for every row in `test.csv`. The dataset is divided into four subproblems, identified by the column `part` $\in \{A, B, C, D\}$. Depending on your modeling strategy, you may either train a single model across all rows or use separate models for different parts.

For model development, use `train.csv` for fitting and `valid.csv` for model selection. After finalizing your approach, retraining on the combined `train.csv+valid.csv` split is allowed before generating predictions for `test.csv`.

The main points to keep in mind are:

- Parts A and B are standard tabular regression tasks.
- Parts C and D are panel-data prediction tasks indexed by `entity_id` and `timestamp`.
- For Parts C and D, the task is **not** a recursive multi-step forecasting rollout. Each test row must be predicted independently using the features already provided for that row.
- Lag variables such as `y_lag1` and `y_lag7`, where available, may be used directly as input features.

Performance is evaluated separately on Parts A–D using mean absolute error (MAE). The final competition score is computed as a weighted combination of the four part-wise MAEs, with lower values indicating better performance.

Your final submission should be a CSV file with exactly two columns:

`row_id,y`

The file must contain exactly one prediction for each row in `test.csv`.

3 Files Provided

The release contains the following CSV files:

- **train.csv**: training split (features + target y).
- **valid.csv**: validation split (features + target y), intended for model selection.
- **test.csv**: test split (features only); targets are withheld for evaluation.
- **sample_submission.csv**: required submission template.

Dataset download (Google Drive):

<https://drive.google.com/drive/u/0/folders/1JURqoe4wNwXWRBTNiFhTOhHgnZ5LMjaO>

Evaluation is reported separately for Parts A–D, but **test.csv** is a single combined table. Your submission must contain predictions for *all* rows in **test.csv**.

Protocol and rules (to avoid leakage).

- **Preprocessing.** Fit imputers, scalers, and encoders on **train** (or **train+valid**) only, and then apply them to **valid/test**.
- **Panel causality.** For a panel row at time t , do not use information from timestamps greater than t .
- **Final training.** After model selection, training on **train+valid** is allowed before producing test predictions.

4 Dataset Schema

All four parts use a common tabular schema, but some columns are only meaningful for certain row types. In particular, the **mode** column indicates whether a row is a standard regression example or a panel time-series example:

- **static**: an ordinary regression row with no entity/time structure. These correspond to Parts A and B.
- **panel_ts**: a panel time-series row indexed by entity and timestamp. These correspond to Parts C and D.

For **static** rows, **entity_id**=-1 and **timestamp**=-1. For **panel_ts** rows, both fields are meaningful and identify the entity and time index of the row.

5 Test Split Summary

The test set contains 26,854 rows with the following part-wise counts:

A: 5,886, B: 4,768, C: 10,800, D: 5,400.

The same test set can also be grouped by row type:

static: 10,654, **panel_ts:** 16,200.

Column	Type	Description
row_id	int/string	Unique identifier used to align predictions with rows.
x1..x20	float	Continuous covariates available in all parts.
z1..z8	int	Discrete covariates in $\{0, \dots, 9\}$.
m1..m6	float	Continuous covariates with missing entries (NaN).
mode	str	Row type: static or panel_ts .
part	str	Evaluation part: A, B, C, or D.
entity_id	int	Panel entity identifier; equals -1 for static rows.
timestamp	int	Discrete time index; equals -1 for static rows.
dow	int	Day-of-week indicator in $\{0, \dots, 6\}$.
month_phase	int	Seasonal bucket in $\{1, 2, 3, 4\}$.
holiday_flag	int	Binary holiday indicator in $\{0, 1\}$.
dept	category	Categorical variable with 25 levels.
building_type	category	Categorical variable with 6 levels.
sensor_class	category	Categorical variable with 4 levels.
student_hint	int	Optional diagnostic covariate; not required for modeling.
y_lag1	float	Lag-1 target for panel rows; may be NaN when undefined.
y_lag7	float	Lag-7 target for panel rows; may be NaN when undefined.
y	float	Target variable (present in train.csv and valid.csv , withheld in test.csv).

Table 1: Dataset schema.

Static rows (Parts A/B)

Parts A and B contain i.i.d. regression rows. For these rows, `mode=static`, `entity_id=-1`, and `timestamp=-1`. The lag features `y_lag1` and `y_lag7` are not defined and therefore appear as NaN.

Panel rows (Parts C/D)

Parts C and D contain panel time-series rows. For these rows, `mode=panel_ts`, `entity_id` identifies the entity, and `timestamp` gives the discrete time index. Lag features may be undefined at early timestamps, so they can contain NaN.

- Part C: timestamps 0–119 over 90 entities; missing-value rate ≈ 0.008 for `y_lag1` and ≈ 0.058 for `y_lag7`.
- Part D: timestamps 120–179 over 90 entities; missing-value rate ≈ 0.000 for `y_lag1` and ≈ 0.000 for `y_lag7`.

6 Evaluation Parts (Modeling Requirements)

Part A: correlated linear regression

Goal: predict a response that is largely explained by a linear relationship under correlated covariates. Appropriate baselines include ridge regression, lasso regression, and regularized tree-based models.

Part B: localized nonlinear interaction

Goal: capture a nonlinear effect that becomes active only in a restricted region of feature space. Methods that model interactions explicitly, such as polynomial features, kernel methods, or boosted trees, can be effective.

Part C: panel forecasting with lag features

Goal: predict the target for each panel row using contemporaneous covariates together with the provided lag variables. This is a row-level supervised prediction task over panel-indexed observations.

Part D: distribution shift and outliers

Goal: remain accurate under covariate shift and heavier-tailed noise or outlier contamination. Robust preprocessing, robust objectives, or outlier-aware strategies may help.

7 Target Definition for Parts C and D

Parts C and D are evaluated as row-level one-step prediction problems. Each panel row is indexed by a pair (e, t) , where:

- e denotes the entity identifier (`entity_id`),
- t denotes the timestamp (`timestamp`).

Thus, (e, t) refers to the row corresponding to entity e at time t in `test.csv`. This is *not* a multi-step rollout forecasting task; all required inputs for that row, including lag features, are already provided.

For each panel row (e, t) in Parts C and D, the prediction target is:

$$\hat{y}_{e,t} = f\left(x_{e,t}, z_{e,t}, m_{e,t}, \text{dept}_{e,t}, \text{building_type}_{e,t}, \text{sensor_class}_{e,t}, t, \text{dow}_{e,t}, \text{month_phase}_{e,t}, \text{holiday_flag}_{e,t}, y_{\text{lag}1_{e,t}}, y_{\text{lag}7_{e,t}}\right). \quad (1)$$

Additional historical features may be constructed from past covariate rows if desired, but withheld test targets must never be used.

8 Scoring

The evaluator computes mean absolute error (MAE) separately for each part and then forms a weighted aggregate. Let T denote the set of all test rows, and let

$$T_p = \{i \in T : \text{part}_i = p\}$$

be the subset of test rows belonging to part p .

Per-part MAE:

$$\text{MAE}_p = \frac{1}{|T_p|} \sum_{i \in T_p} |y_i - \hat{y}_i|, \quad p \in \{A, B, C, D\}. \quad (2)$$

Final score (lower is better):

$$\text{Score} = 0.20 \text{MAE}_A + 0.25 \text{MAE}_B + 0.35 \text{MAE}_C + 0.20 \text{MAE}_D. \quad (3)$$

9 Submission Format

Submit a CSV with exactly two columns:

`row_id, y.`

Your submission must satisfy all of the following:

- every `row_id` from `test.csv` must appear exactly once;
- the predicted value must be provided in the `y` column;
- the row order must remain unchanged from `test.csv`.

Example:

```
row_id,y
test_000001,12.345
test_000002,10.901
...
```

10 Expected Report Content

A strong submission typically includes:

- a baseline model;
- preprocessing choices for missing values and categorical variables;
- a validation protocol using `valid.csv`;
- a part-wise error breakdown;
- one or two lightweight ablations, such as with/without lag features for Parts C/D.